

三個最常見的誤解

做簡報時最值得講的三件事。每一條都可以用這份專案的程式碼當場驗證。

① 「embedding 是資料，不是模型的一部分」

錯。它是權重，跟 `q_proj` 完全平起平坐。

會有這個誤解，是因為 `E` 長得像一張查詢表，直覺會把它歸類成「輸入」。

怎麼當場證明

```
python train.py --steps 1 --trace-steps 1
```

看印出來的參數清單：

```
model.embed.E          204,480    ← 在同一份清單裡
block0.attn.q_proj.W    16,384
block0.ffn.gate_proj.W   43,776
optimizer 拿到 20 個參數張量
```

再看追蹤檔裡兩者的更新：

```
位移 E[...]    -0.000500  +0.000500  +0.000500  -0.000500
位移 q_proj     +0.000500  +0.000500  -0.000500  -0.000500
```

同一個 `optimizer`、同一套規則、同樣的 $\pm lr$ 。

為什麼直覺會漏掉它

換個框架就通了——`E` 就是「第 0 層」：

	輸入	權重	輸出
第 0 層 (embedding)	one-hot (固定資料)	<code>E</code>	詞向量
第 1 層	詞向量	<code>w_q, w_k, w_v</code> , FFN	新的向量

第 1 層的「輸入」就是第 0 層的「輸出」。梯度傳到第 1 層的輸入時不會停在那裡，它繼續往下，變成 `E` 的梯度。

② 「反向傳播會修改權重」

錯。backward() 跑完，權重一個數字都沒變。

```
model.backward(dlogits)    # 只算出「該往哪修」，存進另一個張量
opt.step()                 # 這一行才真的改
```

怎麼當場證明

追蹤檔第 04 節的最後，E 還是原值；到第 06 節 AdamW 那裡才變。

而且方向是相反的，步伐小得多：

更新前 E[...]	-0.01533508	
梯度 g	+0.24444881	
若直接「加上梯度」	+0.22911373	← 錯誤的直覺
AdamW 實際結果	-0.01583501	← 差了 489 倍

兩層都要注意：

1. 梯度是先彼此相加存進梯度表，不是加到 E 上
2. E 真的要改的時候是減，而且要乘上很小的 lr

③ 「token id 是離散的，所以沒辦法微分」

混淆了兩個東西。不對 id 微分——它是資料不是參數。

	是什麼	離散/連續	微分嗎
id = 4	資料	離散	不用，也不能
E[4][0] = -0.0153	參數	連續實數	對它微分

梯度問的是：「把 E[4][0] 動一丁點，L 會變多少？」——跟 id 是不是離散無關。

怎麼當場證明

$f(a, b, c) = a + b$	← c 沒參與
$\partial f / \partial a = 1$	
$\partial f / \partial c = 0$	← 不是「不能微分」，是「算出來就是 0」

寫成矩陣就完全沒有特殊性了：

$x = 0 @ E$ 0 是常數 one-hot 矩陣，E 是變數

$$\partial L / \partial E = O^T \cdot G \quad \leftarrow \text{課本上最普通的矩陣微分}$$

$O^T \cdot G$ 算出來，剛好就是「同一列的相加、沒出現的是 0」—— **scatter-add 不是特殊規則，是矩陣乘法自然的結果。**

有限差分可以直接驗證：

$E[2][0]$ 在	$(L_+ - L_-) / 2\varepsilon = 0.246048$	公式 0.246000	$\leftarrow$ 查兩次，斜率加倍
$E[1][0]$ 狗	$(L_+ - L_-) / 2\varepsilon = 0.000000$	公式 0.000000	$\leftarrow$ 沒被查到

「狗」那一列 $L(w+\varepsilon)$ 和 $L(w-\varepsilon)$ **完全相等**——那個數字沒參與計算，動它 loss 不會變。

延伸：離散在哪裡才真的會咬人

情境	可微嗎
訓練 (teacher forcing)	可微，id 是給定的常數
生成 (argmax / 取樣)	不可微，梯度在這裡斷掉

第二種才是真麻煩——RLHF / GRPO 需要 policy gradient 就是為了繞過它。

附：一個 bonus 誤解

「embedding 的梯度是稀疏的」 —— 在有 weight tying 的模型上**不成立**。

	來源	有幾列非零	
輸入側 (scatter-add)		272 / 3195	$\leftarrow$ 稀疏
輸出側 (weight tying)		3195 / 3195	$\leftarrow$ 稠密
合計		3195 / 3195	

因為 E 同時是輸出層的權重，softmax 要對全部 token 算分數，**每一列每一步都會被碰到**——連完全沒出現在這批資料裡的 token 也一樣。

想親眼看差別：`TinyLM(..., tie_weights=False)` 跑一次，非零列就會剩下 272。